



PROCESAMIENTO DIGITAL DE IMÁGENES

Conversión entre modelos de color

Alumnos:

Enrique Ramírez Pérez, Roberto Misael Reyes Cruz

Introducción:

La conversión entre modelos de color es una técnica fundamental en el procesamiento de imágenes y la representación visual. Los modelos de color son sistemas que describen cómo se percibe y representa el color en un espacio determinado. Cada modelo de color tiene su propia forma de representar los colores y sus componentes, como tono, saturación y brillo. La conversión entre modelos de color permite transformar una imagen de un espacio de color a otro, lo que facilita el análisis, la manipulación y la visualización de las imágenes.

Desarrollo:

Existen varios modelos de color comunes utilizados en el procesamiento de imágenes, como RGB (Rojo, Verde, Azul), HSV (Matiz, Saturación, Valor), CMYK (Cian, Magenta, Amarillo, Negro) y YUV (Luminancia, Crominancia). Cada modelo de color tiene su propio propósito y características únicas.

La conversión entre modelos de color implica la transformación de los valores de color de un modelo a los valores equivalentes en otro modelo. Por ejemplo, para convertir una imagen del modelo RGB al modelo HSV, se aplican cálculos matemáticos para determinar el matiz, la saturación y el valor de cada píxel en el nuevo modelo. Estos cálculos se basan en las relaciones y ecuaciones establecidas para cada modelo de color.

La conversión entre modelos de color es útil en diversas aplicaciones. Por ejemplo, en el campo del procesamiento de imágenes, la conversión a un modelo de color específico puede facilitar la detección de objetos o características basadas en el color. Además, la conversión entre modelos de color puede ser útil para corregir problemas de balance de blancos, ajustar la saturación o brillo de una imagen, o adaptar una imagen a los requisitos de visualización de un dispositivo o medio en particular.

Es importante tener en cuenta que la conversión entre modelos de color puede implicar cierta pérdida o cambio de información, ya que cada modelo de color tiene su propia gama de colores y características. Por lo tanto, es necesario comprender las propiedades y limitaciones de cada modelo de color y considerar el propósito y contexto de la conversión antes de aplicarla.

El mapeo de imagen en escala de grises puede ser útil para corregir errores de exposición, ya que puede mejorar la distribución de la intensidad, aumentar el contraste y hacer que la imagen sea más nítida y fácil de visualizar.

Código

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Cargar la imagen
imagen = cv2.imread('imagencolor.jpeg')

# Convertir la imagen a modelos de color
imagen_xyz = cv2.cvtColor(imagen, cv2.COLOR_BGR2XYZ)

# Calcular los canales de CMY
imagen_cmy = 255 - imagen

# Convertir la imagen a HSV
imagen_hsv = cv2.cvtColor(imagen, cv2.COLOR_BGR2HSV)

# Convertir la imagen a HLS
imagen_hls = cv2.cvtColor(imagen, cv2.COLOR_BGR2HLS_FULL)

# Mostrar las imágenes y sus canales
fig, axs = plt.subplots(5, 4, figsize=(12, 12))

# Mostrar la imagen original
axs[0, 0].imshow(cv2.cvtColor(imagen, cv2.COLOR_BGR2RGB))
axs[0, 0].set_title('Original')
axs[0, 0].axis('off')

# Mostrar los canales de XYZ
for i in range(3):
    axs[1, i].imshow(imagen_xyz[:, :, i], cmap='gray')
    axs[1, i].set_title(f'XYZ {i+1}')
    axs[1, i].axis('off')
```

```

# Mostrar los canales de CMY
for i in range(3):
    axs[2, i].imshow(imagen_cmy[:, :, i])
    axs[2, i].set_title(f'CMY {i+1}')
    axs[2, i].axis('off')

# Mostrar los canales de HSV
for i in range(3):
    if i == 0:
        axs[3, i].imshow(imagen_hsv[:, :, i], cmap='hsv')
    else:
        axs[3, i].imshow(imagen_hsv[:, :, i])
    axs[3, i].set_title(f'HSV {i+1}')
    axs[3, i].axis('off')

# Mostrar los canales de HLS
for i in range(3):
    if i == 0 or i == 1:
        axs[4, i].imshow(imagen_hls[:, :, i], cmap='hsv')
    else:
        axs[4, i].imshow(imagen_hls[:, :, i])
    axs[4, i].set_title(f'HLS {i+1}')
    axs[4, i].axis('off')

plt.tight_layout()
plt.show()

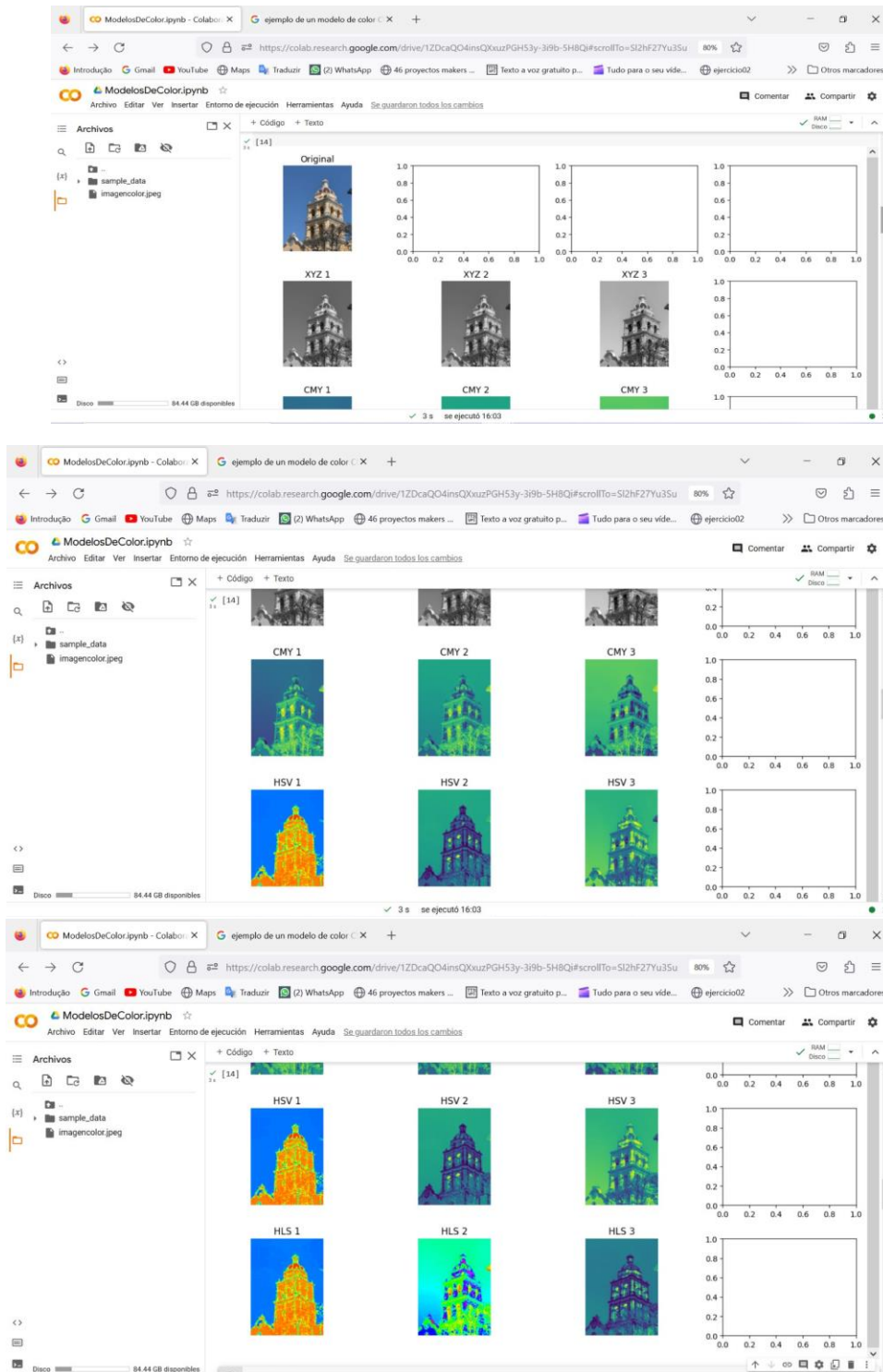
```

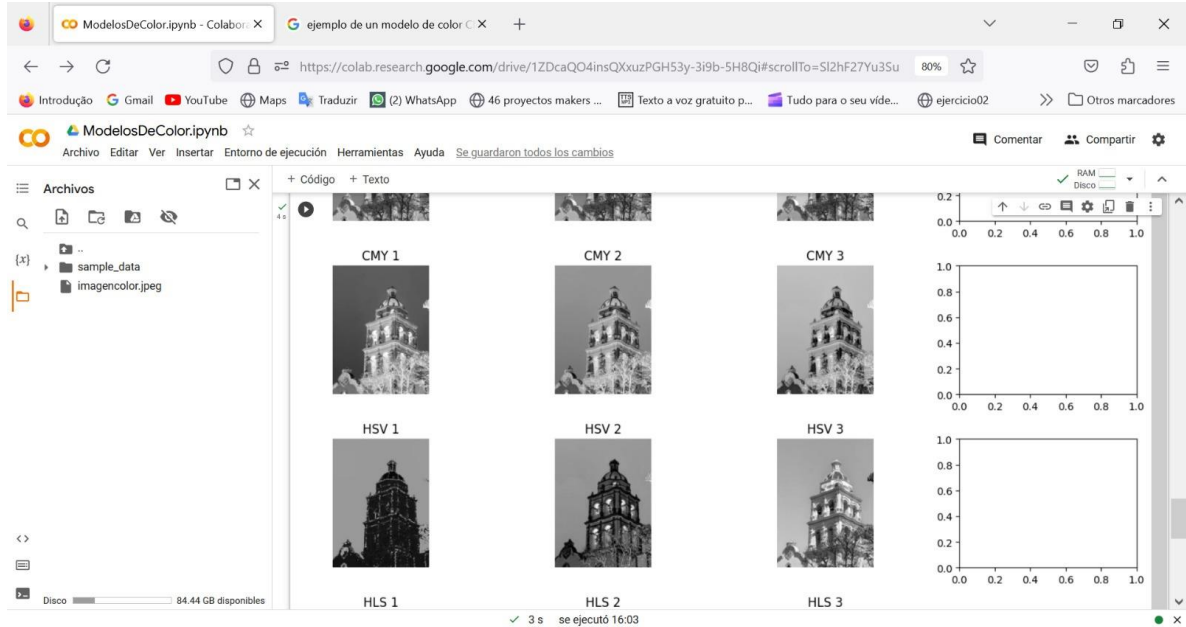
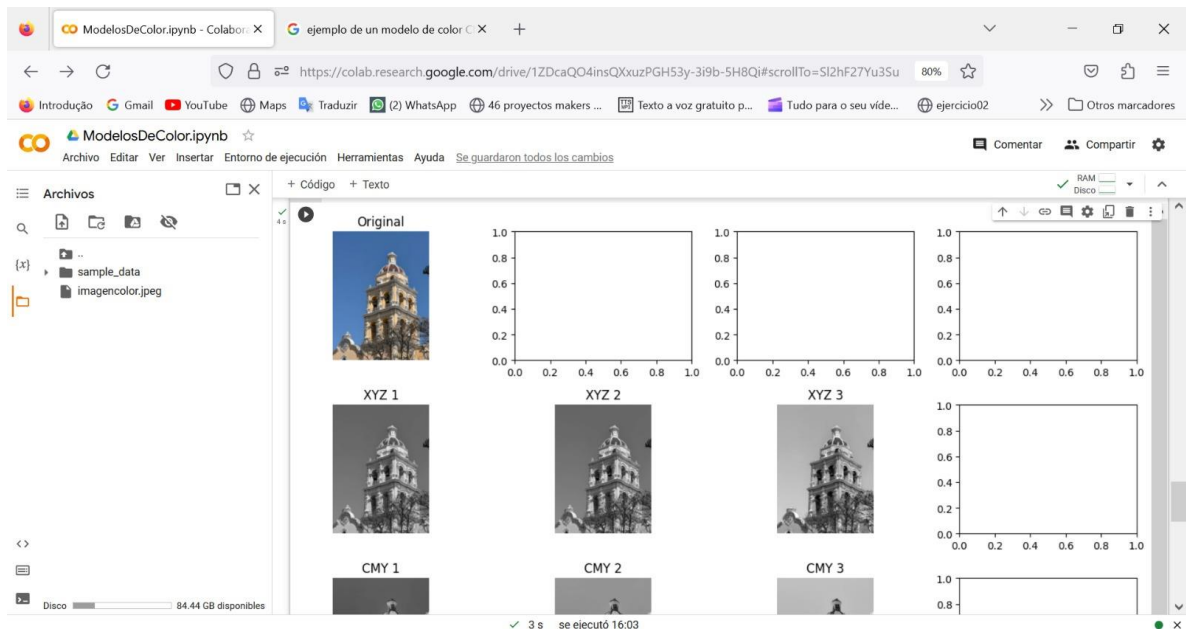
Explicación del código

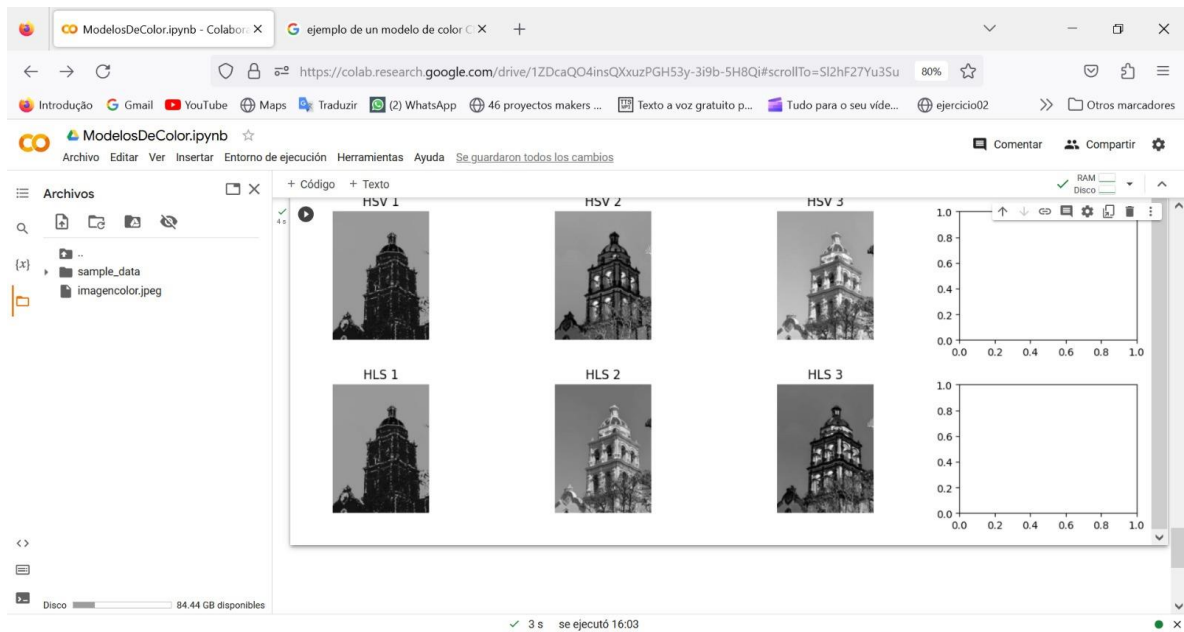
1. Importamos las bibliotecas necesarias: cv2 para el procesamiento de imágenes, numpy para la manipulación de matrices y matplotlib.pyplot para la visualización de imágenes.
2. Cargamos la imagen "imagencolor.jpeg" utilizando la función cv2.imread() y la asignamos a la variable imagen.
3. Convertimos la imagen a un modelo de color XYZ utilizando la función cv2.cvtColor(). El modelo XYZ es un modelo de color basado en la percepción humana del color y es útil para ciertas aplicaciones de procesamiento de imágenes.

4. Calculamos los canales CMY (Cian, Magenta, Amarillo) restando cada componente RGB de 255. Esto invierte los valores de cada canal y representa los colores complementarios.
5. Convertimos la imagen a un modelo de color HSV (Hue, Saturation, Value) utilizando la función `cv2.cvtColor()`. El modelo HSV es útil para el análisis y manipulación de colores, ya que separa el matiz, la saturación y el valor de los píxeles.
6. Convertimos la imagen a un modelo de color HLS (Hue, Lightness, Saturation) utilizando la función `cv2.cvtColor()`. El modelo HLS también es útil para el análisis y manipulación de colores, al separar el matiz, la luminancia y la saturación de los píxeles.
7. Creamos una figura y un conjunto de subplots utilizando la función `plt.subplots()`. Definimos una cuadrícula de 5 filas y 4 columnas para mostrar las imágenes y sus canales correspondientes.
8. Mostramos la imagen original en el primer subplot (fila 0, columna 0) utilizando la función `axs[0, 0].imshow()` y la convertimos a formato RGB utilizando `cv2.cvtColor()`.
9. En los subplots siguientes, mostramos los canales de XYZ en la fila 1 y los canales de CMY en la fila 2. Utilizamos un bucle `for` para iterar sobre los canales y mostrar cada uno en un subplot correspondiente. La función `imshow()` se utiliza para mostrar los canales y `set_title()` para establecer los títulos de los subplots.
10. En los subplots de la fila 3, mostramos los canales de HSV utilizando un enfoque similar al paso anterior. El canal de matiz se muestra con el mapa de colores "hsv" para resaltar su naturaleza cíclica.
11. En los subplots de la fila 4, mostramos los canales de HLS utilizando un enfoque similar al paso anterior. El canal de matiz y el canal de saturación se muestran con el mapa de colores "hsv".
12. Ajustamos el diseño de los subplots utilizando la función `plt.tight_layout()` para evitar superposiciones y asegurar una presentación clara.

13. Finalmente, mostramos la figura con todos los subplots utilizando la función `plt.show()`.







El código muestra la imagen original y los canales de varios modelos de color, lo que nos permite observar cómo se representan y se separan los diferentes componentes de color en cada modelo.

Conclusión

En conclusión, la conversión entre modelos de color es una técnica esencial en el procesamiento de imágenes que permite transformar una imagen de un espacio de color a otro. Esta técnica facilita el análisis, la manipulación y la visualización de imágenes al permitir representar los colores de una manera más adecuada para diferentes aplicaciones y entornos.

Fuentes Bibliograficas

Gonzales, R.C., Woods, R.E. (2008). Digital Image Processing (3rd Edition). Pearson Prentice Hall. - Este libro de referencia es ampliamente utilizado en el campo del procesamiento de imágenes y aborda el tema de la conversión entre modelos de color en detalle.

Foley, J.D., van Dam, A., Feiner, S.K., Hughes, J.F. (1995). Computer Graphics: Principles and Practice (2nd Edition). Addison-Wesley Professional. - Este libro clásico sobre gráficos por computadora ofrece una comprensión sólida de los modelos de color y sus conversiones.

Poynton, C. (2012). Digital Video and HD: Algorithms and Interfaces. Morgan Kaufmann. - Este libro se centra en los aspectos técnicos de la producción de video digital y aborda la conversión entre modelos de color específicamente para aplicaciones de video.